

```

// =====
// Dual-gas PID pressure controller – ZN-optimised
// ESP32 | DRV8825 stepper | DAC valve output | ADC sensors
// =====

// ---- Profile select (A, B, or C) -----
#define PROFILE A

// ---- ADC pins -----
const uint8_t ADC_PIN_AIR = 34;
const uint8_t ADC_PIN_PROP = 35;

// ---- DAC pins (PID output → valve/regulator) -----
const uint8_t DAC_PIN_AIR = 25;
const uint8_t DAC_PIN_PROP = 26;

// ---- Stepper (DRV8825, 200 steps/rev) -----
const int STEP_PIN = 18;
const int DIR_PIN = 19;
const int STEPS_PER_REV = 200;
const float RPM = 8.0;
const unsigned long halfPeriodUs =
    (unsigned long)(6000000.0 / (RPM * STEPS_PER_REV * 2.0));

// ---- Igniter pulse -----
const uint8_t IGNITER_PULSE_PIN = 16;
const unsigned long IGNITER_PULSE_MS = 250;
const unsigned long IGNITER_PERIOD_MS = 1000;

// ---- PID loop interval -----
const unsigned long PID_INTERVAL_MS = 100;

// ---- ADC → pressure scaling -----
// Sensor: 0.25–2.25 V → 0–10 bar
// ESP32 ADC: 0–3.3 V → 0–4095 counts
const float V_LOW = 0.25f;
const float V_HIGH = 2.25f;
const float P_LOW = 0.0f;
const float P_HIGH = 10.0f;
const float ADC_MAX = 4095.0f;
const float VCC = 3.3f;

float adcToPressure(int raw) {
    float v = (raw / ADC_MAX) * VCC;
    return (v - V_LOW) / (V_HIGH - V_LOW) * (P_HIGH - P_LOW) + P_LOW;
}

// ---- Pressure profile arrays -----
// Format: {time_s, air_setpoint_bar, propane_setpoint_bar}
// Generated from sinusoidal profiles A/B/C sampled at 1-second intervals.
// Setpoint = offset + amplitude * sin(2π*t/period + phase), clamped 0–10 bar.

```

```

#define PROFILE_POINTS_A 120
#define PROFILE_POINTS_B 90
#define PROFILE_POINTS_C 60

// Profile A – 120s, slow sinusoid
const float profileA_air[PROFILE_POINTS_A] = {
    3.00,3.24,3.46,3.64,3.78,3.87,3.92,3.93,3.89,3.80,
    3.68,3.53,3.37,3.20,3.04,2.90,2.79,2.72,2.70,2.72,
    2.78,2.88,3.00,3.14,3.28,3.42,3.54,3.63,3.69,3.71,
    3.70,3.65,3.57,3.46,3.34,3.21,3.08,2.97,2.87,2.80,
    2.77,2.77,2.81,2.88,2.97,3.08,3.19,3.30,3.40,3.49,
    3.55,3.59,3.60,3.59,3.55,3.48,3.40,3.30,3.20,3.10,
    3.00,2.91,2.84,2.79,2.77,2.77,2.80,2.85,2.92,3.00,
    3.09,3.18,3.27,3.35,3.42,3.47,3.50,3.51,3.50,3.47,
    3.42,3.35,3.27,3.18,3.09,3.00,2.91,2.83,2.78,2.75,
    2.75,2.77,2.82,2.88,2.96,3.05,3.14,3.23,3.31,3.38,
    3.44,3.48,3.50,3.50,3.48,3.44,3.38,3.31,3.23,3.14,
    3.05,2.96,2.88,2.82,2.77,2.75,2.75,2.78,2.83,2.91
};

const float profileA_prop[PROFILE_POINTS_A] = {
    5.00,5.31,5.60,5.85,6.06,6.21,6.29,6.31,6.26,6.16,
    6.00,5.82,5.61,5.40,5.19,5.00,4.84,4.72,4.65,4.63,
    4.67,4.76,4.90,5.07,5.25,5.43,5.60,5.74,5.84,5.90,
    5.91,5.87,5.79,5.67,5.52,5.36,5.19,5.03,4.89,4.79,
    4.73,4.72,4.76,4.85,4.97,5.12,5.28,5.44,5.58,5.69,
    5.77,5.81,5.80,5.76,5.68,5.57,5.44,5.30,5.15,5.01,
    4.88,4.78,4.72,4.70,4.73,4.79,4.89,5.01,5.15,5.30,
    5.44,5.57,5.68,5.76,5.80,5.81,5.77,5.69,5.58,5.44,
    5.28,5.12,4.97,4.85,4.76,4.72,4.72,4.79,4.89,5.03,
    5.19,5.36,5.52,5.67,5.79,5.87,5.91,5.90,5.84,5.74,
    5.60,5.43,5.25,5.07,4.90,4.76,4.67,4.63,4.65,4.72,
    4.84,5.00,5.19,5.40,5.61,5.82,6.00,6.16,6.26,6.31
};

// Profile B – 90s, faster & phase-shifted
const float profileB_air[PROFILE_POINTS_B] = {
    4.00,4.45,4.83,5.10,5.24,5.24,5.11,4.89,4.61,4.33,
    4.10,3.95,3.92,4.01,4.20,4.46,4.74,4.99,5.17,5.24,
    5.18,5.02,4.78,4.52,4.28,4.12,4.06,4.12,4.29,4.54,
    4.80,5.03,5.19,5.24,5.16,4.98,4.73,4.47,4.24,4.09,
    4.04,4.12,4.30,4.56,4.82,5.04,5.19,5.23,5.14,4.95,
    4.70,4.44,4.22,4.08,4.05,4.14,4.33,4.58,4.84,5.05,
    5.18,5.21,5.11,4.92,4.67,4.41,4.20,4.07,4.06,4.16,
    4.36,4.61,4.86,5.06,5.18,5.19,5.09,4.89,4.64,4.38,
    4.16,4.03,4.03,4.14,4.35,4.61,4.87,5.06,5.17,5.17
};

const float profileB_prop[PROFILE_POINTS_B] = {
    6.00,6.73,7.28,7.56,7.53,7.21,6.69,6.08,5.50,5.05,
    4.82,4.85,5.13,5.60,6.16,6.70,7.11,7.31,7.26,6.97,
    6.49,5.93,5.40,5.00,4.80,4.85,5.14,5.62,6.18,6.71,
    7.10,7.30,7.23,6.93,6.44,5.88,5.36,4.97,4.79,4.86,
    5.17,5.66,6.21,6.73,7.11,7.29,7.21,6.90,6.40,5.83,
    5.32,4.95,4.79,4.88,5.20,5.70,6.24,6.75,7.11,7.27,
    7.18,6.86,6.35,5.79,5.29,4.93,4.79,4.90,5.23,5.73,

```

```

        6.27,6.77,7.11,7.25,7.14,6.81,6.30,5.74,5.25,4.91,
        4.79,4.92,5.26,5.77,6.30,6.79,7.12,7.24,7.12,6.78
    };

// Profile C – 60s, aggressive
const float profileC_air[PROFILE_POINTS_C] = {
    5.50,6.16,6.68,6.95,6.90,6.54,5.96,5.28,4.65,4.21,
    4.06,4.25,4.74,5.39,6.03,6.51,6.74,6.65,6.28,5.70,
    5.04,4.48,4.13,4.10,4.40,4.95,5.61,6.17,6.52,6.57,
    6.29,5.76,5.12,4.55,4.17,4.12,4.42,4.98,5.64,6.19,
    6.53,6.56,6.27,5.73,5.09,4.53,4.16,4.13,4.44,5.01,
    5.67,6.21,6.53,6.55,6.24,5.69,5.05,4.49,4.14,4.13
};

const float profileC_prop[PROFILE_POINTS_C] = {
    7.00,8.09,8.72,8.65,7.96,6.88,5.72,4.79,4.33,4.44,
    5.07,5.99,6.92,7.58,7.77,7.42,6.64,5.63,4.73,4.24,
    4.31,4.92,5.84,6.79,7.49,7.72,7.41,6.65,5.65,4.74,
    4.24,4.31,4.91,5.83,6.78,7.49,7.72,7.42,6.67,5.68,
    4.77,4.26,4.31,4.90,5.81,6.76,7.47,7.71,7.43,6.69,
    5.71,4.80,4.28,4.31,4.88,5.79,6.73,7.45,7.70,7.44
};

// ---- Active profile selection -----
#define _PROFILE_SELECT(P) PROFILE_ ## P
#define _LEN_SELECT(P)     PROFILE_POINTS_ ## P
#define _AIR_SELECT(P)     profile ## P ## _air
#define _PROP_SELECT(P)    profile ## P ## _prop

const float* activeAir    = _AIR_SELECT(PROFILE);
const float* activeProp   = _PROP_SELECT(PROFILE);
const int    profileLen   = _LEN_SELECT(PROFILE);

// ---- ZN-tuned PID -----
// Using "no-overshoot" ZN rules: Kp=0.2Ku, Ti=Tu/2, Td=Tu/3
// Ku and Tu must be identified per rig; placeholders set here.
// To tune: set Kp=Ku, Ki=Kd=0, increase Ku until sustained oscillation, note Tu.

struct PIDState {
    float Kp, Ki, Kd;
    float integral;
    float prevError;
    float outMin, outMax;
};

// Placeholder ZN values – replace Ku/Tu with measured values
const float Ku = 80.0f;    // ultimate gain (DAC units / bar)
const float Tu = 5.0f;     // oscillation period in seconds

PIDState pidAir  = { 0.2f*Ku, (0.2f*Ku)/(0.5f*Tu), (0.2f*Ku)*(Tu/3.0f)/1000.0f, 0,
0, 0, 255 };
PIDState pidProp = { 0.2f*Ku, (0.2f*Ku)/(0.5f*Tu), (0.2f*Ku)*(Tu/3.0f)/1000.0f, 0,
0, 0, 255 };

float pidUpdate(PIDState &s, float setpoint, float measured, float dtSec) {

```

```

float err = setpoint - measured;
s.integral += err * dtSec;
// Anti-windup: clamp integral
float maxInt = s.outMax / (s.Ki > 0 ? s.Ki : 1.0f);
s.integral = constrain(s.integral, -maxInt, maxInt);
float deriv = (err - s.prevError) / dtSec;
s.prevError = err;
float out = s.Kp * err + s.Ki * s.integral + s.Kd * deriv;
return constrain(out, s.outMin, s.outMax);
}

// ---- Pressure → DAC scaling -----
// 0-10 bar maps to DAC 0-255
uint8_t pressureToDac(float bar) {
    return (uint8_t)constrain((bar / P_HIGH) * 255.0f, 0.0f, 255.0f);
}

// ---- Timing state -----
unsigned long lastToggleUs = 0;
unsigned long lastEdgeMs = 0;
unsigned long lastPidMs = 0;
unsigned long profileStartMs = 0;
bool stepState = false;
bool pulseHigh = false;

void setup() {
    Serial.begin(115200);

    pinMode(STEP_PIN, OUTPUT);
    pinMode(DIR_PIN, OUTPUT);
    digitalWrite(DIR_PIN, HIGH);

    pinMode(IGNITER_PULSE_PIN, OUTPUT);
    digitalWrite(IGNITER_PULSE_PIN, LOW);

    dacWrite(DAC_PIN_AIR, 0);
    dacWrite(DAC_PIN_PROP, 0);

    lastToggleUs = micros();
    lastEdgeMs = millis();
    lastPidMs = millis();
    profileStartMs = millis();
}

void loop() {
    unsigned long nowMs = millis();
    unsigned long nowUs = micros();

    // --- Stepper ---
    if (nowUs - lastToggleUs >= halfPeriodUs) {
        stepState = !stepState;
        digitalWrite(STEP_PIN, stepState);
        lastToggleUs = nowUs;
    }
}

```

```
// --- Igniter pulse ---
if (!pulseHigh && (nowMs - lastEdgeMs >= IGNITER_PERIOD_MS -
IGNITER_PULSE_MS)) {
    digitalWrite(IGNITER_PULSE_PIN, HIGH);
    pulseHigh = true;
    lastEdgeMs = nowMs;
} else if (pulseHigh && (nowMs - lastEdgeMs >= IGNITER_PULSE_MS)) {
    digitalWrite(IGNITER_PULSE_PIN, LOW);
    pulseHigh = false;
    lastEdgeMs = nowMs;
}

// --- PID update ---
if (nowMs - lastPidMs >= PID_INTERVAL_MS) {
    float dtSec = (nowMs - lastPidMs) / 1000.0f;
    lastPidMs = nowMs;

    // Profile index (1 sample per second)
    int idx = (int)((nowMs - profileStartMs) / 1000UL);
    idx = constrain(idx, 0, profileLen - 1);

    float spAir = activeAir[idx];
    float spProp = activeProp[idx];

    float measAir = adcToPressure(analogRead(ADC_PIN_AIR));
    float measProp = adcToPressure(analogRead(ADC_PIN_PROP));

    float outAir = pidUpdate(pidAir, spAir, measAir, dtSec);
    float outProp = pidUpdate(pidProp, spProp, measProp, dtSec);

    digitalWrite(DAC_PIN_AIR, (uint8_t)outAir);
    digitalWrite(DAC_PIN_PROP, (uint8_t)outProp);

    // Serial telemetry
    Serial.printf("t=%lus SP_air=%.2f M_air=%.2f OUT_air=%.0f | SP_prop=%.2f
M_prop=%.2f OUT_prop=%.0f\n",
        (nowMs - profileStartMs) / 1000UL,
        spAir, measAir, outAir,
        spProp, measProp, outProp);
}
}
```